

ASK SELF, ASK OTHERS: RELATION IS ALL YOU NEED

Yuting Ge^{1,*}

¹Department of Electrical Engineering
City University of Hong Kong
yutingge3-c@my.cityu.edu.hk

Yang Pengju^{2,†}

²Jilin University
yangda1223@outlook.com

Mingkai Nie^{3,†}

³National University of Singapore
mingkai.nie@u.nus.edu

[†] These authors contributed equally. * Corresponding author.

ABSTRACT

Attention directly derives normalized information flow from pairwise scores. We introduce Relation, an alternative token-mixing primitive that first organizes pairwise evidence into explicit Self and Exchange relations and derives information flow afterward. This relational organization gives rise to Full Relation, FlashRelation, Linear Relation, Hybrid Relation, and a KV-style Relation Cache. Across matched decoder-only models at approximately 10M, 30M, and 100M parameters, Full Relation achieves lower final validation NLL than MHA at all three scales. In a fixed-context reference benchmark, FlashRelation is $3.60\text{--}4.41\times$ faster than the materialized Full Relation implementation. Across scale-matched production workloads, it reaches 76.4–84.9% of PyTorch FlashAttention throughput while executing the Full Relation operator. Hybrid Relation uses 75% Linear Relation layers and achieves strong language-modeling quality. These results support a relation-first view of token mixing: ask Self, ask Others, then let Flow follow Relation.

1 INTRODUCTION

In scaled dot-product Attention, each head maps the interaction between two projected token states to a scalar score (Vaswani et al., 2017),

$$U_{ij} = \frac{q_i^\top k_j}{\sqrt{d_h}}, \quad F_i = \text{Softmax}(U_i).$$

Token interactions in language are heterogeneous. But in Attention, one scalar carries too much. Attention uses the same U_{ij} to encode whatever relational structure matters and to directly determine information flow. Relation formation and flow allocation are therefore compressed into a single $U \rightarrow F$ step.

Attention itself was not always primary. It first appeared as an auxiliary mechanism within recurrent encoder–decoder models before becoming the organizing primitive of the Transformer (Bahdanau et al., 2015; Vaswani et al., 2017). Relation-aware Attention places relation in a similarly subordinate position. Shaw et al. (2018) augment self-attention with pairwise relation representations, but relation is still inside Attention. Attention remains primary: it incorporates relation into its own computation and still determines the flow. We reverse this hierarchy: Flow follows Relation.

We begin by decomposing Relation into two minimal structural roles: Self and Exchange. Self describes a token’s relation with itself, while Exchange describes its relation with other tokens. Pairwise evidence is mapped into these two roles before normalization, and the resulting entries are assembled into a causal Relation matrix R . Normalized information flow F is derived only afterward. This defines the basic Self–Exchange Relation (SER) operator, which extends naturally to Multi-Head Relation (MHR). This explicit organization also exposes further forms of Relation in practice: Full Relation retains explicit token-wise history, FlashRelation factorizes it for tiled execution, Linear Relation compresses historical Others into a recurrent state, Hybrid Relation combines Full and Linear layers, and Relation Cache supports autoregressive decoding.

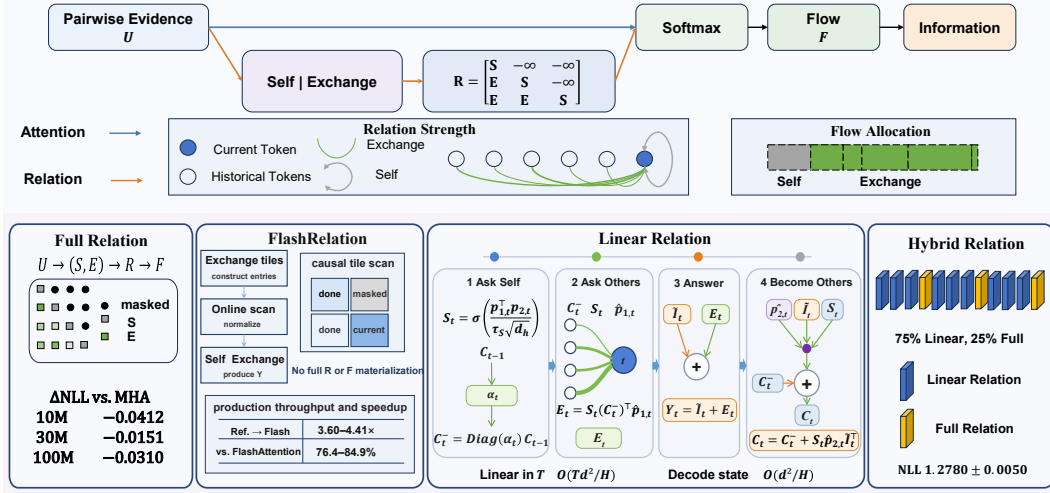


Figure 1: Overview of Relation.

We evaluate Relation at approximately 10M, 30M, and 100M parameters under matched decoder-only settings. Full Relation achieves lower final-validation NLL than MHA at all three scales, while structural ablations examine the roles of its main relational components. We further evaluate practical realizations of Relation: FlashRelation provides exact tiled execution of Full Relation, Linear Relation replaces explicit token-wise history with a recurrent relational state, Hybrid Relation composes Full and Linear layers, and Relation Cache supports autoregressive decoding. Together, these results support Relation as a viable and practical alternative token-mixing primitive (Figure 1). Our main contributions are:

- We introduce Self–Exchange Relation (SER) and Multi-Head Relation (MHR) as Relation operators for token mixing.
- We characterize Self–Exchange flow and derive its exact factorization into Self mass, aggregate Exchange mass, and allocation within historical Exchange.
- Building on Relation’s distinct relational organization, we develop Relation-specific practical forms, including FlashRelation, Linear Relation, and Hybrid Relation, and mathematically derive an exact KV-style Relation Cache for autoregressive decoding.

2 BACKGROUND

Scaled dot-product Attention represents pairwise token interactions through compatibility scores that are normalized into token-to-token mixing weights (Vaswani et al., 2017). Subsequent work has enriched how these interactions are represented and transformed. Relative-position Attention incorporates pairwise relation representations into the scoring process (Shaw et al., 2018), while learned or structured positional biases reshape pre-normalization scores (Raffel et al., 2020; Press et al., 2022; Chi et al., 2022). Sigmoid self-attention modifies the normalization itself (Ramapuram et al., 2025). Talking-Heads Attention introduces projections across the attention-head dimension around Softmax (Shazeer et al., 2020), while the Gated Attention Unit combines single-head attention with gating (Hua et al., 2022).

Beyond Attention, alternative token-mixing operators explore different ways to organize information exchange across a sequence. FNet replaces self-attention with Fourier mixing (Lee-Thorp et al., 2022), PoNet uses multi-granularity pooling (Tan et al., 2022), and HyperMixer dynamically constructs token-mixing MLPs (Mai et al., 2023).

A separate line of work focuses on how token mixing is executed and how historical information is represented. FlashAttention and FlashAttention-2 evaluate exact softmax Attention through tiled, IO-aware execution (Dao et al., 2022; Dao, 2024). Linear and recurrent approaches replace explicit

quadratic token-token interaction with associative or recurrent states (Katharopoulos et al., 2020; Sun et al., 2023; Yang et al., 2024; 2025). Kimi Delta Attention (KDA) further develops recurrent-state token mixing, while Kimi Linear combines KDA with full-attention MLA layers in a hybrid architecture (Kimi Team et al., 2025).

These lines place Relation alongside three complementary directions in token mixing: interaction representation, efficient execution, and recurrent history. Relation connects these directions through a common relational organization. The following sections develop this framework from its basic Self-Exchange Relation (SER) operator to its practical forms.

3 RELATION

We first introduce Self-Exchange Relation (SER), the basic Relation operator. We then extend SER to Multi-Head Relation (MHR), as illustrated in Figure 2.

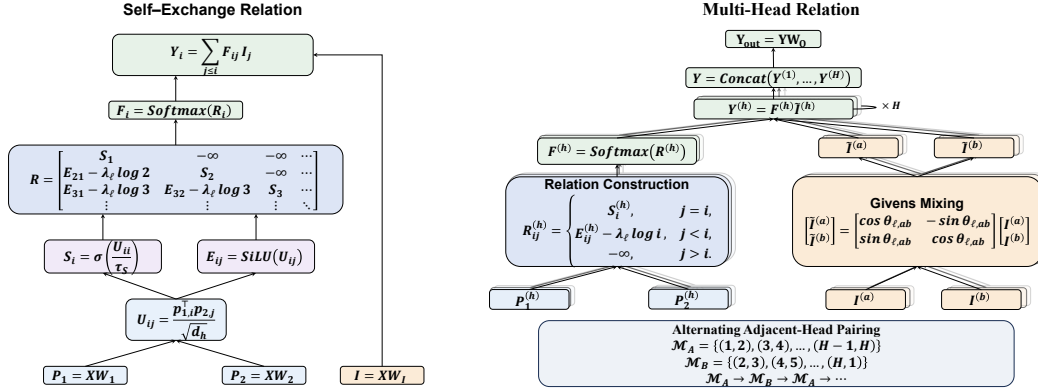


Figure 2: Self-Exchange Relation and Multi-Head Relation.

3.1 SELF-EXCHANGE RELATION

Given an input sequence X , we project it into two relation spaces and an information space,

$$P_1 = XW_1, \quad P_2 = XW_2, \quad I = XW_I.$$

$$U_{ij} = \frac{p_{1,i}^\top p_{2,j}}{\sqrt{d_h}}. \quad (1)$$

The head-wise P_1 and P_2 entering U use RoPE, while I remains unrotated.

Relation distinguishes a token’s interaction with itself from its interactions with other tokens. We define the Self and Exchange relation mappings as

$$S_i = \sigma \left(\frac{U_{ii}}{\tau_S} \right), \quad E_{ij} = \text{SiLU}(U_{ij}), \quad (2)$$

$$R_{ij} = \begin{cases} S_i, & j = i, \\ E_{ij} - \lambda_\ell \log i, & j < i, \\ -\infty, & j > i. \end{cases}$$

The Self temperature $\tau_S > 0$ controls the scale and smoothness of the bounded Self mapping. We set $\tau_S = 2$ in all formal experiments and this fixed value is not treated as theoretically unique.

As the number of visible predecessors increases, aggregate Exchange evidence receives a candidate-count contribution. We therefore use a learnable layer-wise coefficient λ_ℓ and apply the logarithmic

correction $-\lambda_\ell \log i$, following the broader use of logarithmic length corrections in attention mechanisms (Ramapuram et al., 2025).

Normalized information flow is derived only after the Relation has been constructed and is applied to the information states,

$$\begin{aligned} F_i &= \text{Softmax}(R_i), \\ Y_i &= \sum_{j \leq i} F_{ij} I_j, \\ X_{\ell+1} &= X_\ell + Y W_O. \end{aligned} \tag{3}$$

3.2 MULTI-HEAD RELATION

We extend Relation to H heads. The projected relation spaces and information space are partitioned into H subspaces, each of width $d_h = d/H$. Each head independently applies the SER construction from Sec. 3.1, yielding its own $R^{(h)}$ and $F^{(h)}$.

Before applying the flow, we mix the information states across adjacent head pairs using learnable Givens rotations (Jing et al., 2017). The pairing pattern alternates between successive layers,

$$\begin{bmatrix} \tilde{I}^{(a)} \\ \tilde{I}^{(b)} \end{bmatrix} = \begin{bmatrix} \cos \theta_{\ell,ab} & -\sin \theta_{\ell,ab} \\ \sin \theta_{\ell,ab} & \cos \theta_{\ell,ab} \end{bmatrix} \begin{bmatrix} I^{(a)} \\ I^{(b)} \end{bmatrix}. \tag{4}$$

Each head then transports its mixed information according to its own flow, and the head outputs are concatenated and projected back to the residual stream,

$$\begin{aligned} Y^{(h)} &= F^{(h)} \tilde{I}^{(h)}, \\ X_{\ell+1} &= X_\ell + \text{Concat}_{h=1}^H \left(Y^{(h)} \right) W_O. \end{aligned} \tag{5}$$

4 WHY RELATION

Why introduce an explicit Relation matrix between pairwise evidence and information flow? The purpose of R is to change how token mixing is organized before normalization.

4.1 RELATION IS CONSTRUCTED BEFORE FLOW

In canonical Attention, pairwise compatibility scores directly form the logits from which normalized information flow is derived. Relation instead constructs an explicit relational structure before normalization. As defined in Sec. 3.1, R is assembled from role-specific Self and Exchange entries, with Self on the diagonal and Exchange over the causal history. The two roles are therefore defined before they compete for information flow.

Treating R as an explicit object in the operator definition does not require materializing the full matrix during execution. Because R is defined entrywise from pairwise evidence through the Self–Exchange construction, the required evidence and Exchange entries can be formed and consumed as needed. R specifies how token mixing is organized, while its computational representation can remain implicit.

4.2 HOW RELATION FORMS SELF–EXCHANGE FLOW

Because Self and Exchange are already explicit in R , the normalized flow admits an exact decomposition aligned with these roles. For token i , collect the historical Exchange entries into

For token $i > 1$, let S_i denote the constructed Self relation from Eq. (2). We define the historical Exchange log-normalizer L_i^E , its count-corrected aggregate A_i , and the full row normalizer Z_i as

$$\begin{aligned}
\text{LSE}_{j<i}(a_j) &\equiv \log \sum_{j<i} e^{a_j}, \\
L_i^E &= \text{LSE}_{j<i}(E_{ij}), \quad A_i = L_i^E - \lambda_\ell \log i = \text{LSE}_{j<i}(R_{ij}), \\
Z_i &= e^{S_i} + e^{A_i}.
\end{aligned} \tag{6}$$

The normalized flow first divides its mass between Self and aggregate Exchange,

$$F_{ii} = \frac{e^{S_i}}{e^{S_i} + e^{A_i}} = \sigma(S_i - A_i), \quad g_i \equiv \sum_{j<i} F_{ij} = \frac{e^{A_i}}{e^{S_i} + e^{A_i}} = \sigma(A_i - S_i), \quad F_{ii} + g_i = 1. \tag{7}$$

Thus, the normalized flow is partitioned exactly between the Self mass F_{ii} and the aggregate Exchange mass g_i .

Conditional on Exchange, the mass g_i is distributed across historical tokens according to

$$\begin{aligned}
\pi_{ij}^E &\equiv \frac{F_{ij}}{g_i} = \frac{e^{R_{ij}}}{\sum_{k<i} e^{R_{ik}}} = \frac{e^{E_{ij}}}{\sum_{k<i} e^{E_{ik}}}, \quad j < i, \\
F_{ij} &= \frac{e^{R_{ij}}}{Z_i} = \frac{e^{A_i}}{Z_i} \frac{e^{R_{ij}}}{e^{A_i}} = g_i \frac{e^{R_{ij}}}{\sum_{k<i} e^{R_{ik}}} = g_i \pi_{ij}^E.
\end{aligned} \tag{8}$$

The communicated information is therefore

$$Y_i = \sum_{j \leq i} F_{ij} \tilde{I}_j = F_{ii} \tilde{I}_i + g_i \sum_{j<i} \pi_{ij}^E \tilde{I}_j. \tag{9}$$

The normalized flow therefore has two levels. It first determines how much mass remains with Self and how much enters Exchange. Conditional on Exchange, it then determines how that mass is distributed across historical tokens. Intuitively, Relation first asks whether a token should rely on itself or draw from its history, and if it draws from history, it then decides where to look.

4.3 INSIDE RELATION: HOW A TOKEN'S FLOW IS DETERMINED

We now examine what determines these two stages for an individual token i . The balance between Self and aggregate Exchange is given exactly by

$$\log \frac{g_i}{F_{ii}} = A_i - S_i = \text{LSE}_{j<i}(E_{ij}) - S_i - \lambda_\ell \log i. \tag{10}$$

Equation (10) shows that the Self-Exchange balance is governed exactly by the gap $A_i - S_i$.

A larger $A_i - S_i$ shifts flow toward Exchange, whereas a smaller $A_i - S_i$ shifts flow toward Self.

Once flow enters Exchange, the question changes from whether to use history to where in history to look. For any two historical positions $j, k < i$,

$$R_{ij} - R_{ik} = E_{ij} - E_{ik}. \tag{11}$$

Because the logarithmic correction is shared by all historical positions in the row, it cancels in pairwise differences. The relative allocation within Exchange is therefore determined entirely by the Exchange relations E_{ij} .

Thus, for token i , Self versus aggregate Exchange is governed by $A_i - S_i$, while allocation within history is governed by relative Exchange relations.

4.4 WHAT RELATION CHANGES IN TOKEN MIXING

In Relation, Self and Exchange are explicitly constructed before normalization. Their distinct roles survive in the resulting information flow, even after normalization. Canonical Attention has no such relational stage: its pairwise scores are normalized directly into flow. Relation therefore makes relation formation a distinct stage of token mixing, with information flow derived afterward. We next turn from this relational organization to its practical realizations and evaluation in Sections 5 and 6.

5 RELATION IN PRACTICE

The same Self–Exchange organization gives rise to a family of practical Relation forms. We call the explicit token-to-token operator defined in Sec. 3 Full Relation. FlashRelation is its exact fused execution, while Linear Relation represents historical Relation through a recurrent state. Hybrid Relation composes Full and Linear layers within a single decoder, and Relation Cache provides the autoregressive state used by Full Relation.

5.1 FACTORIZED SELF–EXCHANGE: FLASHRELATION

The Relation factorization derived in Sec. 4.2 separates historical Exchange aggregation from the final allocation between Self and Exchange. This factorization leads us to develop FlashRelation, a tiled realization of Full Relation. For token i , historical Exchange entries are processed through a causal tiled scan. For each tile $\mathcal{B}$, we maintain a running maximum m , normalizer l , and information accumulator z :

$$\begin{aligned} m' &= \max \left(m, \max_{j \in \mathcal{B}} E_{ij} \right), \\ l' &= e^{m-m'} l + \sum_{j \in \mathcal{B}} e^{E_{ij}-m'}, \quad z' = e^{m-m'} z + \sum_{j \in \mathcal{B}} e^{E_{ij}-m'} \tilde{I}_j. \end{aligned} \quad (12)$$

After each tile, $(m, l, z) \leftarrow (m', l', z')$. Let (m_i, l_i, z_i) denote the terminal accumulators after the historical Exchange scan. They yield

$$L_i^E = m_i + \log l_i, \quad \bar{I}_i^E = \frac{z_i}{l_i}, \quad (13)$$

where L_i^E is the aggregate historical Exchange log-normalizer and $\bar{I}_i^E$ is the normalized historical information summary. The row is then completed by

$$A_i = L_i^E - \lambda_\ell \log i, \quad g_i = \sigma(A_i - S_i), \quad Y_i = (1 - g_i) \tilde{I}_i + g_i \bar{I}_i^E. \quad (14)$$

Following the online-softmax tiling principle of FlashAttention (Dao et al., 2022), the historical Exchange scan is evaluated tile by tile without materializing the full R or F .

5.2 CHANGE EXCHANGE: LINEAR AND HYBRID RELATION

Full Relation represents Exchange explicitly between the current token and individual historical tokens. Exchange can instead operate over a recurrent relational state that compresses individual historical Others, while Self remains tied to the current token. This leads to Linear Relation, with

$$C_t^{(h)} \in \mathbb{R}^{d_h \times d_h}.$$

Using the same projections introduced in Sec. 3.1, the current Self relation and the two normalized relation coordinates are

$$S_t = \sigma \left(\frac{p_{1,t}^\top p_{2,t}}{\tau_S \sqrt{d_h}} \right), \quad (15)$$

$$\hat{p}_{1,t} = \frac{p_{1,t}}{\|p_{1,t}\|_2}, \quad \hat{p}_{2,t} = \frac{p_{2,t}}{\|p_{2,t}\|_2}.$$

Historical Relation is maintained with an input-dependent channel-wise retention. Linear Relation applies the same RoPE to $p_{1,t}$ and $p_{2,t}$ before computing S_t and the normalized relation coordinates. Following the retention parameterization of KDA (Kimi Team et al., 2025),

$$r_t = W_\alpha^\uparrow W_\alpha^\downarrow \bar{x}_t, \quad g_t = -\exp(A_{\log}) \odot \text{softplus}(r_t + dt_{\text{bias}}), \quad \alpha_t = \exp(g_t) \in (0, 1]^{d_h}. \quad (16)$$

With historical Others compressed into a recurrent state, the current token determines how that history is used through its Self relation. Linear Relation therefore follows the sequence

$$\boxed{\text{Ask Self} \rightarrow \text{Ask Others} \rightarrow \text{Answer} \rightarrow \text{Become Others}}.$$

The recurrent Relation step is

$$\begin{aligned} C_t^- &= \text{Diag}(\alpha_t) C_{t-1}, \\ E_t &= S_t (C_t^-)^\top \hat{p}_{1,t}, \\ Y_t &= \tilde{I}_t + E_t, \\ C_t &= C_t^- + S_t \hat{p}_{2,t} \tilde{I}_t^\top. \end{aligned} \quad (17)$$

Here, S_t determines the current Self relation and modulates both the read from historical Others and the write that makes the current token part of future Others. The read occurs before the current write, so E_t has strictly historical support. Each token asks Self, consults Others, forms its answer, and then becomes Others for future tokens. The resulting token-mixing complexity and decode-state requirements are summarized in Table 1.

Table 1: Complexity refers to the token-mixing core. Linear sequential operations refer to the direct recurrence.

Layer Type	Token-Mixing Complexity	Sequential Ops.	Decode History State
MHA / FlashAttention	$O(T^2 d)$	$O(1)$	$O(Td)$
Full / FlashRelation	$O(T^2 d)$	$O(1)$	$O(Td)$
Linear Relation	$\mathbf{O}(T \mathbf{d}^2 / \mathbf{H})$	$O(T)$	$\mathbf{O}(\mathbf{d}^2 / \mathbf{H})$

Full and Linear Relation provide complementary regimes. Linear Relation scales linearly with sequence length and uses a fixed-size decode state, while Full Relation retains $O(1)$ sequential operations. Hybrid Relation combines the two by interleaving Linear and Full Relation layers. Following the hybrid layerwise design used in Kimi Linear (Kimi Team et al., 2025), our evaluated configuration uses nine Linear Relation layers and three Full Relation layers.

5.3 CACHE HISTORY: RELATION CACHE

During autoregressive decoding, Attention commonly caches the key and value states of previous tokens to avoid recomputing the full prefix (Shazeer, 2019; Pope et al., 2023). Relation admits an analogous projected cache. At each step, only the new Relation row is formed. From the pairwise-evidence definition in Sec. 3.1, each future row uses its own current P_1 coordinate against historical P_2 coordinates. Once a token’s row has been formed, its P_1 state is therefore no longer reused, whereas its P_2 state continues to participate in Relation construction for future tokens. Historical information states are likewise reused for transport. Therefore,

$$\mathcal{C}_t = \{P_{2,1:t}, \tilde{I}_{1:t}\}. \quad (18)$$

At the next decoding step, only the new Relation row is constructed from the current P_1 and the cached historical states.

6 EXPERIMENTS

We compare MHA and Full Relation under matched decoder geometry, data order, and training budget. Experiments cover approximately 10M, 30M, and 100M parameters and use paired seeds 42, 43, and 44. Scale-specific architectures, datasets, context lengths, and token budgets are summarized in Table 2. Our primary metric is final-checkpoint NLL on the full validation set, and structural ablations use the 10M setting with the same three seeds. All models use full-head RoPE with base 10^4 and no positional scaling. Systems benchmarks are conducted separately on an NVIDIA RTX 5090 in BF16 and report tuned steady-state throughput. Full optimization, evaluation, per-seed, and secondary benchmark details are provided in Appendices B, C, and E.

6.1 LANGUAGE MODELING AND STRUCTURAL ABLATIONS

Across the three model scales, Full Relation achieves lower mean final-validation NLL than MHA, with improvements of 0.0412, 0.0151, and 0.0310 at 10M, 30M, and 100M parameters, respectively (Table 2).

Table 2: Main validation results. NLL is mean $\pm$ sample standard deviation over three seeds. Backbone reports layers / d_{model} / heads $\times d_h$ / d_{ff} .

Scale	Backbone	Data/Vocab	Ctx	Tokens	MHA NLL $\downarrow$	Relation NLL $\downarrow$	Δ
10M	6L/384/8 $\times$ 48/768	TinyStories/4K	1024	150M	1.6853 $\pm$ 0.0042	1.6441 $\pm$ 0.0124	-0.0412
30M	10L/512/8 $\times$ 64/1024	TinyStories/4K	2048	450M	1.3001 $\pm$ 0.0044	1.2850 $\pm$ 0.0136	-0.0151
100M	20L/640/8 $\times$ 80/1280	SmolLM/32K	4096	1.071B	2.9373 $\pm$ 0.0093	2.9063 $\pm$ 0.0061	-0.0310

We next examine several structural choices in Full Relation on the 10M setting (Table 3). Exchange-only transport and Raw-X communication increase final-validation NLL, while removing count calibration or collapsing Relation to a single head produces larger degradations. We also evaluate No-Givens mixing ($\tilde{I} = I$). The exact ablation definitions and per-seed results are provided in Appendix D.

Table 3: Structural ablations on the 10M setting. NLL is mean $\pm$ sample standard deviation over three seeds.

Metric	Full Relation	Exchange-only transport $Y_i = \sum_{j < i} F_{ij} \tilde{I}_j$	Raw-X communication $I = X$	No count calibration $\lambda_\ell = 0$	No Givens mixing $\tilde{I} = I$	Single head $H = 1$
NLL $\downarrow$	1.6441 $\pm$ 0.0124	1.6761 $\pm$ 0.0097	1.6807 $\pm$ 0.0074	1.6947 $\pm$ 0.0203	1.6474 $\pm$ 0.0176	1.6948 $\pm$ 0.0221
Δ vs Full	—	+0.0320	+0.0366	+0.0506	+0.0032	+0.0507

6.2 EFFICIENT IMPLEMENTATIONS

In a fixed-context $T = 1024$ reference benchmark, FlashRelation is $3.60\text{--}4.41\times$ faster than the materialized Full Relation reference implementation. Across the three scale-matched production workloads, FlashRelation reaches 76.4–84.9% of PyTorch FlashAttention throughput while executing the exact Full Relation operator (Table 4). Same-micro throughput comparisons and Linear Relation throughput results are reported in Appendices H and I.

Table 4: FlashRelation systems results on an RTX 5090 in BF16. (a) Fixed-context ($T = 1024$) comparison with the materialized Full Relation reference. (b) Scale-matched production throughput against PyTorch FlashAttention.

(a) Fixed-context $T = 1024$				
Scale	Reference Full	FlashRelation	Speedup	
10M	128,515	560,909	$4.36\times$	
30M	74,376	327,927	$4.41\times$	
100M	30,631	110,311	$3.60\times$	

(b) Scale-matched production				
Scale	Context	FlashAttention	FlashRelation	FR / FA
10M	1024	841,834	672,838	$0.799\times$
30M	2048	361,793	307,146	$0.849\times$
100M	4096	107,783	82,333	$0.764\times$

Hybrid Relation combines nine Linear and three Full Relation layers in a $(LLLF)^3$ layout. Under the 30M-class training setting, it achieves 1.2780 ± 0.0050 final-validation NLL across three seeds (Table 5). This demonstrates that Linear and Full Relation layers can be composed in a decoder in which nine of twelve token-mixing layers use Linear Relation.

Table 5: Hybrid Relation under the 30M-class training setting.

Model	Backbone	Layout	Params	NLL $\downarrow$
Hybrid Relation	12L/480/8 $\times$ 60/960	$(LLLF)^3$	31.97M	1.2780 ± 0.0050

7 LIMITATIONS AND CONCLUSION

Limitations. Our experiments cover decoder-only language models up to approximately 100M parameters and training budgets up to 1.071B tokens. Linear and Hybrid Relation are evaluated in selected configurations, while Relation Cache is established mathematically. Behavior at substantially larger scales and in multimodal or post-training settings remains open.

Conclusion. We introduced Relation as an alternative token-mixing primitive in which pairwise evidence is first organized into explicit Self and Exchange relations, and information flow is derived afterward. Within this framework, Full Relation retains explicit token-wise history, while FlashRelation provides exact factorized execution. Linear Relation compresses historical Others into a recurrent state, and Hybrid Relation combines Full and Linear layers. A KV-style Relation Cache follows for autoregressive decoding. Across three model scales, Full Relation achieves lower final validation NLL than matched MHA baselines, while FlashRelation provides practical execution and Hybrid Relation retains strong language-modeling quality with 75% Linear Relation layers. Together, these results support a broader view of token mixing. Token mixing need not be organized directly around information flow: it can begin with Relation.

REFERENCES

- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1409.0473>.
- Ta-Chung Chi, Ting-Han Fan, Peter J. Ramadge, and Alexander Rudnicky. Kerple: Kernelized relative positional embedding for length extrapolation. In *Advances in Neural Information Processing Systems*, volume 35, pp. 8386–8399, 2022. doi: 10.52202/068431-0610. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/37a413841a614b5414b333585e7613b8-Abstract-Conference.html.
- Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, volume 2024, pp. 35549–35562,

-
2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/file/98ed250b203dlac6b24bbcf263e3d4a7-Paper-Conference.pdf.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pp. 16344–16359, 2022. doi: 10.52202/068431-1189. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.
- Weizhe Hua, Zihang Dai, Hanxiao Liu, and Quoc Le. Transformer quality in linear time. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162, pp. 9099–9117. PMLR, 2022. URL <https://proceedings.mlr.press/v162/hua22a.html>.
- Li Jing, Yichen Shen, Tena Dubcek, John Peurifoy, Scott Skirlo, Yann LeCun, Max Tegmark, and Marin Soljačić. Tunable efficient unitary neural networks (EUNN) and their application to RNNs. In Doina Precup and Yee Whye Teh (eds.), *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 1733–1741. PMLR, 06–11 Aug 2017. URL <https://proceedings.mlr.press/v70/jing17a.html>.
- Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119, pp. 5156–5165. PMLR, 2020. URL <https://proceedings.mlr.press/v119/katharopoulos20a.html>.
- Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, Wentao Li, Enzhe Lu, Weizhou Liu, Yanru Chen, Weixin Xu, Longhui Yu, Yejie Wang, Yu Fan, Longguang Zhong, Enming Yuan, Dehao Zhang, Yizhi Zhang, T. Y. Liu, Haiming Wang, Shengjun Fang, Weiran He, Shaowei Liu, Yiwei Li, Jianlin Su, Jiezhong Qiu, Bo Pang, Junjie Yan, Zhejun Jiang, Weixiao Huang, Bohong Yin, Jiacheng You, Chu Wei, Zhengtao Wang, Chao Hong, Yutian Chen, Guanduo Chen, Yucheng Wang, Huabin Zheng, Feng Wang, Yibo Liu, Mengnan Dong, Zheng Zhang, Siyuan Pan, Wenhao Wu, Yuhao Wu, Longyu Guan, Jiawen Tao, Guohong Fu, Xinran Xu, Yuzhi Wang, Guokun Lai, Yuxin Wu, Xinyu Zhou, Zhilin Yang, and Yulun Du. Kimi linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*, 2025. URL <https://arxiv.org/abs/2510.26692>.
- James Lee-Thorp, Joshua Ainslie, Ilya Eckstein, and Santiago Ontañón. Fnet: Mixing tokens with fourier transforms. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4296–4313. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.naacl-main.319. URL <https://aclanthology.org/2022.naacl-main.319/>.
- Florian Mai, Arnaud Pannatier, Fabio Fehr, Haolin Chen, Francois Marelli, Francois Fleuret, and James Henderson. Hypermixer: An mlp-based low cost alternative to transformers. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15632–15654. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.871. URL <https://aclanthology.org/2023.acl-long.871/>.
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, volume 5, pp. 606–624, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html.
- Ofir Press, Noah Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=R8sQPpGCv0>.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL <https://jmlr.org/papers/v21/20-074.html>.

-
- Jason Ramapuram, Federico Danieli, Eeshan Gunesh Dhekane, Floris Weers, Dan Busbridge, Pierre Ablin, Tatiana Likhomanenko, Jagrit Digani, Zijin Gu, Amitis Shidani, and Russell Webb. Theory, analysis, and best practices for sigmoid self-attention. In *International Conference on Learning Representations*, volume 2025, pp. 65464–65519, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/a43b459e9fab3a703148ba0c83b8a442-Paper-Conference.pdf.
- Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pp. 464–468, 2018. doi: 10.18653/v1/N18-2074. URL <https://aclanthology.org/N18-2074/>.
- Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019. URL <https://arxiv.org/abs/1911.02150>.
- Noam Shazeer, Zhenzhong Lan, Youlong Cheng, Nan Ding, and Le Hou. Talking-heads attention. *arXiv preprint arXiv:2003.02436*, 2020. URL <https://arxiv.org/abs/2003.02436>.
- Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023. URL <https://arxiv.org/abs/2307.08621>.
- Chao-Hong Tan, Qian Chen, Wen Wang, Qinglin Zhang, Siqi Zheng, and Zhen-Hua Ling. Ponet: Pooling network for efficient token mixing in long sequences. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=9jInD9JjicF>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pp. 5998–6008, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
- Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 56501–56523. PMLR, 2024. URL <https://proceedings.mlr.press/v235/yang24ab.html>.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations*, pp. 29687–29707, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/4904fad153f6434a7bcf04465d4be2cc-Abstract-Conference.html.

A DETAILED OPERATOR DERIVATIONS

This appendix records the operator definitions used throughout the paper. Token indices in the count correction are one-based. A head has width d_h , and the information state after input projection and head mixing is denoted by $\tilde{I}$.

A.1 SELF-EXCHANGE RELATION

A.1.1 RELATION CONSTRUCTION

We first write the construction for one head. If the model-wide projection has width d , the following quantities are understood after selecting one head subspace of width $d_h = d/H$. Thus

$$P_1, P_2, I \in \mathbb{R}^{T \times d_h}, \quad P_1 = XW_1, \quad P_2 = XW_2, \quad I = XW_I.$$

The head-wise P_1 and P_2 entering U use full-head RoPE, whereas I remains unrotated. Writing $p_{1,i}$ and $p_{2,j}$ for rows of these projected states,

$$U_{ij} = \frac{p_{1,i}^\top p_{2,j}}{\sqrt{d_h}}.$$

The Self and Exchange entries are

$$S_i = \sigma\left(\frac{U_{ii}}{\tau_S}\right), \quad E_{ij} = \text{SiLU}(U_{ij}), \quad \tau_S > 0, \quad \tau_S = 2.$$

Here τ_S is the Self temperature used to set the scale and smoothness of the bounded Self mapping. The count-calibration parameter λ_ℓ is one unconstrained FP32 scalar per layer, shared across heads and initialized to 0.5. Causality is imposed when these role-specific entries are assembled:

$$R_{ij} = \begin{cases} S_i, & j = i, \\ E_{ij} - \lambda_\ell \log i, & j < i, \\ -\infty, & j > i. \end{cases}$$

A.1.2 EXACT ROW NORMALIZER

The normalized row is $F_i = \text{Softmax}(R_i)$. For $i > 1$, its normalizer is

$$\begin{aligned} Z_i &= \sum_{k \leq i} e^{R_{ik}} \\ &= e^{S_i} + \sum_{j < i} e^{E_{ij} - \lambda_\ell \log i} \\ &= e^{S_i} + e^{-\lambda_\ell \log i} \sum_{j < i} e^{E_{ij}} \\ &= e^{S_i} + i^{-\lambda_\ell} \sum_{j < i} e^{E_{ij}}. \end{aligned}$$

Define the historical Exchange log-normalizer and its count-corrected version by

$$L_i^E = \text{LSE}_{j < i}(E_{ij}) = \log \sum_{j < i} e^{E_{ij}}, \quad A_i = L_i^E - \lambda_\ell \log i.$$

Then

$$e^{A_i} = e^{-\lambda_\ell \log i} \sum_{j < i} e^{E_{ij}}, \quad \boxed{Z_i = e^{S_i} + e^{A_i}}.$$

A.1.3 SELF MASS AND AGGREGATE EXCHANGE MASS

The diagonal Self probability is

$$\begin{aligned} F_{ii} &= \frac{e^{S_i}}{Z_i} = \frac{e^{S_i}}{e^{S_i} + e^{A_i}} \\ &= \frac{1}{1 + e^{A_i - S_i}} = \boxed{\sigma(S_i - A_i)}. \end{aligned}$$

The total historical Exchange mass is

$$\begin{aligned} g_i &= \sum_{j < i} F_{ij} = \frac{e^{A_i}}{e^{S_i} + e^{A_i}} \\ &= \frac{1}{1 + e^{S_i - A_i}} = \boxed{\sigma(A_i - S_i)}. \end{aligned}$$

Consequently,

$$\boxed{F_{ii} + g_i = 1}.$$

The Self-Exchange odds are

$$\frac{g_i}{F_{ii}} = \frac{e^{A_i}}{e^{S_i}} = e^{A_i - S_i},$$

and therefore

$$\boxed{\log \frac{g_i}{F_{ii}} = A_i - S_i = L_i^E - S_i - \lambda_\ell \log i.}$$

A.1.4 CONDITIONAL EXCHANGE ALLOCATION

For a historical position $j < i$, define its conditional Exchange allocation as

$$\pi_{ij}^E = \frac{F_{ij}}{g_i}.$$

Substituting the normalized row and the aggregate Exchange mass gives

$$\begin{aligned} \pi_{ij}^E &= \frac{e^{E_{ij} - \lambda_\ell \log i} / Z_i}{e^{A_i} / Z_i} \\ &= \frac{e^{E_{ij} - \lambda_\ell \log i}}{\sum_{k < i} e^{E_{ik} - \lambda_\ell \log i}} \\ &= \frac{e^{E_{ij}}}{\sum_{k < i} e^{E_{ik}}}. \end{aligned}$$

The row-level count correction cancels because it is identical for every historical position. Hence

$$\boxed{\pi_{ij}^E = \text{Softmax}_{j < i}(E_{ij})}, \quad \sum_{j < i} \pi_{ij}^E = 1,$$

and the original historical flow entries factor as

$$\boxed{F_{ij} = g_i \pi_{ij}^E}.$$

A.1.5 COUNT CORRECTION

For any two historical positions $j, k < i$,

$$\begin{aligned} R_{ij} - R_{ik} &= (E_{ij} - \lambda_\ell \log i) - (E_{ik} - \lambda_\ell \log i) \\ &= \boxed{E_{ij} - E_{ik}}. \end{aligned}$$

Equivalently,

$$\frac{\pi_{ij}^E}{\pi_{ik}^E} = e^{E_{ij} - E_{ik}}.$$

Thus $-\lambda_\ell \log i$ is a row-level Exchange translation. It changes the Self-Exchange mass competition through $A_i - S_i$, but it does not change conditional Exchange allocation.

A.1.6 EXACT INFORMATION TRANSPORT

Define the normalized historical information summary

$$\bar{I}_i^E = \sum_{j < i} \pi_{ij}^E \tilde{I}_j.$$

The original Full Relation transport is

$$\begin{aligned} Y_i &= \sum_{j \leq i} F_{ij} \tilde{I}_j \\ &= F_{ii} \tilde{I}_i + \sum_{j < i} F_{ij} \tilde{I}_j \\ &= F_{ii} \tilde{I}_i + g_i \sum_{j < i} \pi_{ij}^E \tilde{I}_j \\ &= \boxed{(1 - g_i) \tilde{I}_i + g_i \bar{I}_i^E}. \end{aligned}$$

This is an exact algebraic factorization of the original Full Relation transport, not a new operator. The residual output remains $X_{\ell+1} = X_\ell + YW_O$.

A.1.7 BOUNDARY CASE

For the first token, the historical set is empty. Define

$$\text{LSE}_{j < 1} = -\infty, \quad A_1 = -\infty.$$

It follows directly that

$$\boxed{g_1 = 0, \quad F_{11} = 1, \quad Y_1 = \tilde{I}_1}.$$

A.2 MULTI-HEAD RELATION

A.2.1 HEADWISE CONSTRUCTION

Let $d_h = d/H$. After projection and head splitting,

$$P_1^{(h)}, P_2^{(h)}, I^{(h)} \in \mathbb{R}^{T \times d_h}, \quad h = 1, \dots, H.$$

Each head independently applies the Self-Exchange construction of Appendix A.1, yielding

$$R^{(h)}, \quad F^{(h)}, \quad Y^{(h)}.$$

Relation construction is head-local and cross-head interaction is applied only to the information branch.

A.2.2 GIVENS BLOCK

For an adjacent head pair (a, b) , define

$$G(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

The information states are mixed by

$$\begin{bmatrix} \tilde{I}^{(a)} \\ \tilde{I}^{(b)} \end{bmatrix} = G(\theta_{\ell, ab}) \begin{bmatrix} I^{(a)} \\ I^{(b)} \end{bmatrix}.$$

Since

$$G(\theta)^\top G(\theta) = I_2,$$

the transformation is orthogonal. Applied to every information channel of the pair,

$$\|\tilde{I}^{(a)}\|_2^2 + \|\tilde{I}^{(b)}\|_2^2 = \|I^{(a)}\|_2^2 + \|I^{(b)}\|_2^2.$$

The same scalar angle $\theta_{\ell, ab}$ acts on the d_h information channels associated with the pair.

A.2.3 ALTERNATING PAIRING

The formal construction uses an even number of heads, and in all formal experiments, $H = 8$. With one-based head indices, the two pairings are

$$\mathcal{M}_A = \{(1, 2), (3, 4), \dots, (H-1, H)\}, \quad \mathcal{M}_B = \{(2, 3), (4, 5), \dots, (H, 1)\}.$$

They alternate across layers:

$$\mathcal{M}_A \longrightarrow \mathcal{M}_B \longrightarrow \mathcal{M}_A \longrightarrow \dots$$

For $H = 8$,

$$\mathcal{M}_A = \{(1, 2), (3, 4), (5, 6), (7, 8)\}, \quad \mathcal{M}_B = \{(2, 3), (4, 5), (6, 7), (8, 1)\}.$$

There are $H/2 = 4$ learnable angles per layer. They are stored in FP32 and initialized to zero.

A.2.4 MULTI-HEAD TRANSPORT

After the role-specific flow is formed for each head,

$$Y^{(h)} = F^{(h)} \tilde{I}^{(h)}.$$

The head outputs are concatenated and projected:

$$Y = \text{Concat} \left(Y^{(1)}, \dots, Y^{(H)} \right), \quad X_{\ell+1} = X_\ell + Y W_O.$$

Thus the Relation construction remains head-local, Givens rotations act on the information branch, and W_O combines the head outputs.

A.3 FLASHRELATION

A.3.1 HISTORICAL EXCHANGE INVARIANT

FlashRelation evaluates the exact factorization of Appendix A.1 without materializing the full $T \times T$ matrices. For a fixed query row i , let $\mathcal{J} \subseteq \{1, \dots, i-1\}$ be the historical indices already scanned. The historical Exchange state is defined by the invariant

$$m = \max_{j \in \mathcal{J}} E_{ij}, \quad l = \sum_{j \in \mathcal{J}} e^{E_{ij}-m}, \quad z = \sum_{j \in \mathcal{J}} e^{E_{ij}-m} \tilde{I}_j.$$

This scan contains Exchange entries only and Self is not included in (m, l, z) . For the empty set,

$$m = -\infty, \quad l = 0, \quad z = 0.$$

A.3.2 ADDING ONE TILE

Let $\mathcal{B}$ be the next causal tile and set

$$m_{\mathcal{B}} = \max_{j \in \mathcal{B}} E_{ij}, \quad m' = \max(m, m_{\mathcal{B}}).$$

The previously scanned terms are rescaled into the new maximum:

$$\begin{aligned} \sum_{j \in \mathcal{J}} e^{E_{ij}-m'} &= e^{m-m'} \sum_{j \in \mathcal{J}} e^{E_{ij}-m} \\ &= e^{m-m'} l. \end{aligned}$$

Consequently, the updated normalizer is

$$l' = e^{m-m'} l + \sum_{j \in \mathcal{B}} e^{E_{ij}-m'}.$$

The same rescaling gives the information accumulator

$$z' = e^{m-m'} z + \sum_{j \in \mathcal{B}} e^{E_{ij}-m'} \tilde{I}_j.$$

Together with $m' = \max(m, m_{\mathcal{B}})$, these updates preserve the invariant with $\mathcal{J} \leftarrow \mathcal{J} \cup \mathcal{B}$.

A.3.3 TERMINAL ACCUMULATORS

After all historical positions $j < i$ have been scanned,

$$m_i = \max_{j < i} E_{ij}, \quad l_i = \sum_{j < i} e^{E_{ij} - m_i}, \quad z_i = \sum_{j < i} e^{E_{ij} - m_i} \tilde{I}_j.$$

The terminal log-normalizer is

$$\begin{aligned} m_i + \log l_i &= m_i + \log \sum_{j < i} e^{E_{ij} - m_i} \\ &= \log \left(e^{m_i} \sum_{j < i} e^{E_{ij} - m_i} \right) \\ &= \log \sum_{j < i} e^{E_{ij}} \\ &= \boxed{L_i^E}. \end{aligned}$$

A.3.4 HISTORICAL INFORMATION SUMMARY

The terminal accumulator gives

$$\begin{aligned} \frac{z_i}{l_i} &= \frac{\sum_{j < i} e^{E_{ij} - m_i} \tilde{I}_j}{\sum_{k < i} e^{E_{ik} - m_i}} \\ &= \frac{\sum_{j < i} e^{E_{ij}} \tilde{I}_j}{\sum_{k < i} e^{E_{ik}}} \\ &= \sum_{j < i} \pi_{ij}^E \tilde{I}_j. \end{aligned}$$

The common factor e^{-m_i} cancels. Therefore

$$\boxed{\frac{z_i}{l_i} = \bar{I}_i^E}.$$

A.3.5 RECOVERING THE FULL RELATION ROW

The row-level count correction is applied after the historical scan:

$$A_i = m_i + \log l_i - \lambda_\ell \log i, \quad g_i = \sigma(A_i - S_i).$$

The final transport is

$$Y_i = (1 - g_i) \tilde{I}_i + g_i \frac{z_i}{l_i}.$$

Using the identities in Appendix A.1,

$$\boxed{Y_i^{\text{FlashRelation}} = Y_i^{\text{FullRelation}}}.$$

So FlashRelation changes the execution of the exact operator.

A.3.6 MATERIALIZATION AND BOUNDARY

FlashRelation does not materialize full $T \times T$ U , R , or F matrices. The required pairwise evidence and Exchange entries are formed and consumed tile by tile. The historical state is maintained as running statistics.

A.4 LINEAR RELATION

A.4.1 RELATION COORDINATES AND RETENTION

In the equations below, $p_{1,t}$ and $p_{2,t}$ denote the projected relation coordinates after the formal RoPE step. The positional order is

$$P_1/P_2 \longrightarrow \text{RoPE} \longrightarrow S_t \longrightarrow \text{L2Norm} \longrightarrow \text{recurrence}.$$

Thus

$$S_t = \sigma \left(\frac{p_{1,t}^\top p_{2,t}}{\tau_S \sqrt{d_h}} \right), \quad \hat{p}_{1,t} = \frac{p_{1,t}}{\|p_{1,t}\|_2}, \quad \hat{p}_{2,t} = \frac{p_{2,t}}{\|p_{2,t}\|_2}.$$

The retention parameterization is

$$r_t = W_\alpha^\uparrow W_\alpha^\downarrow \bar{x}_t, \quad g_t = -\exp(A_{\log}) \odot \text{softplus}(r_t + dt_{\text{bias}}), \quad \alpha_t = \exp(g_t).$$

Because $\exp(A_{\log}) > 0$ elementwise and $\text{softplus}(\cdot) > 0$,

$$g_t < 0, \quad \boxed{0 < \alpha_{t,c} \leq 1}.$$

Here S_t is the Self Relation, whereas α_t is channel-wise historical retention.

A.4.2 RECURRENT STEP

Let

$$D_t = \text{Diag}(\alpha_t), \quad \boxed{C_0 = 0}.$$

The recurrent step is

$$\begin{aligned} C_t^- &= D_t C_{t-1}, \\ E_t &= S_t (C_t^-)^\top \hat{p}_{1,t}, \\ Y_t &= \tilde{I}_t + E_t, \\ C_t &= C_t^- + S_t \hat{p}_{2,t} \tilde{I}_t^\top. \end{aligned}$$

The execution order is

$$\boxed{\text{Ask Self} \longrightarrow \text{Ask Others} \longrightarrow \text{Answer} \longrightarrow \text{Become Others.}}$$

A.4.3 STRICT HISTORICAL SUPPORT

The read uses $C_t^- = D_t C_{t-1}$, while the current write

$$S_t \hat{p}_{2,t} \tilde{I}_t^\top$$

is added only after E_t has been formed. Hence E_t depends only on positions $1, \dots, t-1$, and the current token cannot enter its own historical read through the current write.

A.4.4 UNROLLING THE RECURRENT STATE

Starting from

$$C_t = D_t C_{t-1} + S_t \hat{p}_{2,t} \tilde{I}_t^\top,$$

one expansion gives

$$\begin{aligned} C_t &= D_t \left(D_{t-1} C_{t-2} + S_{t-1} \hat{p}_{2,t-1} \tilde{I}_{t-1}^\top \right) + S_t \hat{p}_{2,t} \tilde{I}_t^\top \\ &= D_t D_{t-1} C_{t-2} + D_t S_{t-1} \hat{p}_{2,t-1} \tilde{I}_{t-1}^\top + S_t \hat{p}_{2,t} \tilde{I}_t^\top. \end{aligned}$$

With the ordered product

$$\prod_{u=j+1}^t D_u \equiv D_t D_{t-1} \cdots D_{j+1},$$

and the empty product equal to the identity,

$$\boxed{C_t = \sum_{j \leq t} \left(\prod_{u=j+1}^t D_u \right) S_j \hat{p}_{2,j} \tilde{I}_j^\top.}$$

Thus each token contributes $S_j \hat{p}_{2,j} \tilde{I}_j^\top$ and is subsequently transformed by $D_{j+1}, \dots, D_t$.

A.4.5 EXPANDED HISTORICAL EXCHANGE

The pre-write state is

$$C_t^- = \sum_{j < t} \left(\prod_{u=j+1}^t D_u \right) S_j \hat{p}_{2,j} \tilde{I}_j^\top.$$

Substituting this into the read gives

$$\begin{aligned} E_t &= S_t (C_t^-)^\top \hat{p}_{1,t} \\ &= \sum_{j < t} S_t S_j \left[\hat{p}_{2,j}^\top \left(\prod_{u=j+1}^t D_u \right) \hat{p}_{1,t} \right] \tilde{I}_j. \end{aligned}$$

Because each D_u is diagonal, the transpose leaves the ordered product unchanged.

A.4.6 INFORMATION WRITE AND COMPLEXITY

The output is

$$Y_t = \tilde{I}_t + E_t,$$

but the state writes

$$S_t \hat{p}_{2,t} \tilde{I}_t^\top,$$

not $S_t \hat{p}_{2,t} Y_t^\top$. Each new state contribution therefore corresponds to the current token's own information rather than recursively storing the information already read from history.

For one head, $C_t^{(h)} \in \mathbb{R}^{d_h \times d_h}$. The principal recurrent operations per token are

$$D_t C_{t-1}, \quad (C_t^-)^\top \hat{p}_{1,t}, \quad \hat{p}_{2,t} \tilde{I}_t^\top,$$

with leading cost $O(d_h^2)$ per token per head. Across H heads,

$$O(THd_h^2) = O\left(TH \left(\frac{d}{H}\right)^2\right) = \boxed{O\left(\frac{Td^2}{H}\right)}.$$

The recurrent state size is

$$Hd_h^2 = H \left(\frac{d}{H}\right)^2 = \frac{d^2}{H}, \quad \boxed{O(d^2/H)}$$

for the decode state. These complexities refer to the recurrent Relation token-mixing core and exclude common FFN and embedding costs.

A.5 RELATION CACHE

A.5.1 WHAT THE NEXT ROW NEEDS

At autoregressive position $t + 1$, the new pairwise evidence row is

$$U_{t+1,j} = \frac{p_{1,t+1}^\top p_{2,j}}{\sqrt{d_h}}, \quad j \leq t + 1.$$

For historical positions $j \leq t$, the new row needs the cached $p_{2,j}$ states. Historical $p_{1,j}$ states do not participate in future row construction, so $P_{1,1:t}$ need not be retained.

A.5.2 WHAT TRANSPORT NEEDS

The new position transports

$$Y_{t+1} = \sum_{j \leq t+1} F_{t+1,j} \tilde{I}_j.$$

Thus the historical information states $\tilde{I}_{1:t}$ must also remain available. The exact projected cache is

$$\boxed{\mathcal{C}_t = \{P_{2,1:t}, \tilde{I}_{1:t}\}}.$$

A.5.3 CACHE UPDATE

After position $t + 1$ has been processed, its projected historical coordinate and information state are appended:

$$\mathcal{C}_{t+1} = \mathcal{C}_t \cup \left\{ P_{2,t+1}, \tilde{I}_{t+1} \right\}.$$

At the following step, only the new current states $P_{1,t+2}$, $P_{2,t+2}$, and $\tilde{I}_{t+2}$ are formed, and the historical states $P_{2,1:t+1}$, $\tilde{I}_{1:t+1}$ are read from the cache.

A.5.4 OBJECTS NOT CACHED AND CACHE SIZE

The exact autoregressive dependency does not require retaining

$$P_{1,1:t}, \quad U_{1:t,1:t}, \quad R_{1:t,1:t}, \quad F_{1:t,1:t}.$$

Only P_2 and $\tilde{I}$ are needed as historical projected states for the next row and its transport. The projected cache stores two d -wide states per historical token, so its size grows as

$$O(Td).$$

B EXPERIMENTAL SETUP AND EVALUATION

B.1 MODEL CONFIGURATIONS

All models use full-head RoPE with base 10^4 . The three main scales use paired seeds 42, 43, 44.

Table 6: Formal decoder-only model configurations. Global tokens per optimizer update are 131,072 for all three scales.

Scale	Params MHA/Rel.	$L/d/H \times d_h/d_{ff}$	Vocab	Data	T	Train	Micro/GA
10M	10,425,216/10,425,246	6/384/8 $\times$ 48/768	4K	TinyStories	1024	150M	32/4
30M	28,322,304/28,322,354	10/512/8 $\times$ 64/1024	4K	TinyStories	2048	450M	16/4
100M	102,917,760/102,917,860	20/640/8 $\times$ 80/1280	32K	SmolLM corpus	4096	1.071B	MHA 2/16; Rel. 4/8

B.2 OPTIMIZATION AND CHECKPOINTS

Training uses BF16 and AdamW with betas (0.9, 0.95), weight decay 0.1, gradient clipping at 1.0, and a WSD80 schedule. Learning rates are 10^{-3} , 8×10^{-4} , and 6×10^{-4} for 10M, 30M, and 100M. Warmup tokens are 1.5M, 4.5M, and 10.7114M. Formal NLL is evaluated on the full validation set at the final checkpoint.

B.3 POSITION ENCODING

RoPE covers the full head dimension. The base is 10^4 . For MHA, RoPE is applied to Q and K after projection and head splitting. For Full Relation and FlashRelation, RoPE is applied to P_1 and P_2 before pairwise evidence. The information projection and Givens output remain unrotated. Linear Relation applies RoPE to its two relation coordinates before Self and normalized coordinate formation. Hybrid layers use the corresponding Full or Linear path.

B.4 EVALUATION PROTOCOL

The primary metric is final-checkpoint NLL. Formal BLiMP uses 67 categories and 67,000 examples per seed. Secondary evaluations use the final checkpoints and the same three seeds. Systems measurements report complete optimizer-step throughput. One-time initialization and JIT, warmup, and terminal partial steps are excluded when present.

C PER-SEED LANGUAGE-MODEL RESULTS

C.1 FINAL VALIDATION NLL

Table 7 reports final validation NLL for all paired seeds. Aggregate rows use the sample standard deviation over three seeds.

Table 7: Final validation NLL for the three paired seeds.

Scale	Seed	MHA NLL	Full Relation NLL	Full-MHA
10M	42	1.688823	1.631738	-0.057085
10M	43	1.686405	1.643978	-0.042427
10M	44	1.680626	1.656633	-0.023993
10M	Mean $\pm$ SD	1.685285 ± 0.004212	1.644117 ± 0.012448	-0.041168
30M	42	1.295263	1.300730	+0.005467
30M	43	1.303885	1.277177	-0.026708
30M	44	1.301160	1.277225	-0.023935
30M	Mean $\pm$ SD	1.300103 ± 0.004407	1.285044 ± 0.013584	-0.015059
100M	42	2.945050	2.912234	-0.032816
100M	43	2.939826	2.899965	-0.039861
100M	44	2.926899	2.906657	-0.020242
100M	Mean $\pm$ SD	2.937258 ± 0.009344	2.906285 ± 0.006143	-0.030973

C.2 AGGREGATED LANGUAGE-MODEL RESULTS

Full Relation wins all three paired comparisons at 10M, two of three at 30M, and all three at 100M, for eight wins out of nine paired comparisons.

D STRUCTURAL ABLATION DETAILS

D.1 ABLATION DEFINITIONS

All structural ablations use the 10M setting, the same prepared data, and seeds 42, 43, and 44. Each ablation changes only the named component of Full Relation.

- Exchange-only transport computes the complete Full Relation flow and transports only historical entries: $Y_i = \sum_{j < i} F_{ij} \tilde{I}_j$. Historical weights are not renormalized.
- Raw- X communication replaces the independent information projection by $I = X$. Head splitting, Givens mixing, Flow, and W_O remain unchanged.
- No count calibration sets $\lambda_\ell = 0$.
- No Givens mixing bypasses adjacent-head rotation, so $\tilde{I} = I$, while the Relation and transport path remains unchanged.
- Single head uses $H = 1$.

D.2 PER-SEED ABLATION RESULTS

Table 8: Per-seed final validation NLL for the structural ablations.

Ablation	Seed 42	Seed 43	Seed 44	Mean $\pm$ SD	Δ vs Full
Full Relation	1.631738	1.643978	1.656633	1.644117 ± 0.012448	0.000000
Exchange-only transport	1.665658	1.677815	1.684822	1.676099 ± 0.009697	+0.031982
Raw- X communication	1.672332	1.686600	1.683090	1.680674 ± 0.007434	+0.036557
No count calibration	1.671373	1.704531	1.708256	1.694720 ± 0.020305	+0.050604
No Givens mixing	1.630321	1.646350	1.665386	1.647352 ± 0.017554	+0.003236
Single head	1.669349	1.707688	1.707422	1.694820 ± 0.022059	+0.050703

E SECONDARY BENCHMARKS

The secondary evaluations below use final checkpoints and the same three seeds as the language-model experiments. Accuracy is a fraction of examples. For LAMBADA, perplexity and mean target NLL are lower-is-better metrics.

E.1 AGGREGATED RESULTS

Table 9: Formal BLiMP results over 67 categories and 67,000 examples per seed.

Scale	Seed	MHA BLiMP	Full Relation BLiMP	Full-MHA
10M	42	0.572567	0.588746	+0.016179
10M	43	0.572299	0.581418	+0.009119
10M	44	0.571746	0.579463	+0.007716
10M	Mean $\pm$ SD	0.572204 $\pm$ 0.000419	0.583209 $\pm$ 0.004894	+0.011005
30M	42	0.598075	0.614806	+0.016731
30M	43	0.621657	0.618134	-0.003522
30M	44	0.622328	0.625806	+0.003478
30M	Mean $\pm$ SD	0.614020 $\pm$ 0.013813	0.619582 $\pm$ 0.005641	+0.005562
100M	42	0.750433	0.745955	-0.004478
100M	43	0.770299	0.766030	-0.004269
100M	44	0.765090	0.749403	-0.015687
100M	Mean $\pm$ SD	0.761940 $\pm$ 0.010300	0.753796 $\pm$ 0.010734	-0.008144

Table 10: Aggregated results for overall-accuracy task families.

Task	Scale	MHA mean $\pm$ SD	MHR mean $\pm$ SD	MHR-MHA	Wins	n
blimp_supplement	10M	0.570190 $\pm$ 0.013869	0.569460 $\pm$ 0.012034	-0.000730	2	3
blimp_supplement	30M	0.553051 $\pm$ 0.016250	0.572745 $\pm$ 0.014946	0.019693	3	3
blimp_supplement	100M	0.578655 $\pm$ 0.014846	0.594045 $\pm$ 0.009545	0.015390	3	3
comps	10M	0.530813 $\pm$ 0.012635	0.539954 $\pm$ 0.002803	0.009141	2	3
comps	30M	0.558654 $\pm$ 0.009019	0.556384 $\pm$ 0.008536	-0.002270	0	3
comps	100M	0.628584 $\pm$ 0.004908	0.615241 $\pm$ 0.019606	-0.013343	0	3
entity_tracking	10M	0.210176 $\pm$ 0.058427	0.229008 $\pm$ 0.045517	0.018832	2	3
entity_tracking	30M	0.363665 $\pm$ 0.050151	0.335445 $\pm$ 0.080940	-0.028219	2	3
entity_tracking	100M	0.291394 $\pm$ 0.030456	0.328333 $\pm$ 0.009184	0.036939	3	3

Overall-accuracy task families.

Table 11: Aggregated B-group metrics across the three seeds.

Task	Scale	Split	Metric	MHA mean $\pm$ SD	MHR mean $\pm$ SD	MHR-MHA	Wins	n
arc_challenge	10M	-	acc	0.208473 $\pm$ 0.012662	0.201784 $\pm$ 0.011745	-0.006689	1	3
arc_challenge	10M	-	acc_norm	0.261984 $\pm$ 0.020435	0.254181 $\pm$ 0.017697	-0.007804	0	3
arc_challenge	30M	-	acc	0.210702 $\pm$ 0.010033	0.198439 $\pm$ 0.006962	-0.012263	1	3
arc_challenge	30M	-	acc_norm	0.247492 $\pm$ 0.008849	0.221851 $\pm$ 0.010218	-0.025641	0	3
arc_challenge	100M	-	acc	0.212932 $\pm$ 0.013517	0.222965 $\pm$ 0.007724	0.010033	3	3
arc_challenge	100M	-	acc_norm	0.241918 $\pm$ 0.006962	0.237458 $\pm$ 0.005793	-0.004459	0	3
openbookqa	10M	-	acc	0.131333 $\pm$ 0.007572	0.127333 $\pm$ 0.013013	-0.004000	1	3
openbookqa	10M	-	acc_norm	0.169333 $\pm$ 0.011719	0.164000 $\pm$ 0.007211	-0.005333	1	3
openbookqa	30M	-	acc	0.120000 $\pm$ 0.005292	0.112000 $\pm$ 0.010583	-0.008000	1	3
openbookqa	30M	-	acc_norm	0.174000 $\pm$ 0.015620	0.170000 $\pm$ 0.012000	-0.004000	1	3
openbookqa	100M	-	acc	0.162667 $\pm$ 0.005033	0.157333 $\pm$ 0.004619	-0.005333	1	3
openbookqa	100M	-	acc_norm	0.286000 $\pm$ 0.011136	0.286000 $\pm$ 0.003464	-0.000000	1	3
lambada_standard	10M	test	accuracy	0.038812 $\pm$ 0.003498	0.045669 $\pm$ 0.003888	0.006857	3	3
lambada_standard	10M	test	perplexity	539.084666 $\pm$ 194.121401	488.769003 $\pm$ 25.590314	-50.315663	2	3
lambada_standard	10M	test	mean_target_nll	6.247946 $\pm$ 0.351631	6.190993 $\pm$ 0.051624	-0.056953	2	3
lambada_standard	10M	validation	accuracy	0.038817 $\pm$ 0.004040	0.048470 $\pm$ 0.003651	0.009653	3	3
lambada_standard	10M	validation	perplexity	575.877730 $\pm$ 215.622529	525.337625 $\pm$ 38.883730	-50.540104	1	3
lambada_standard	10M	validation	mean_target_nll	6.311539 $\pm$ 0.359425	6.262198 $\pm$ 0.074529	-0.049341	1	3
lambada_standard	30M	test	accuracy	0.070315 $\pm$ 0.001954	0.066951 $\pm$ 0.000846	-0.003364	0	3
lambada_standard	30M	test	perplexity	133.042252 $\pm$ 22.776937	202.971570 $\pm$ 68.930202	69.929318	0	3
lambada_standard	30M	test	mean_target_nll	4.881056 $\pm$ 0.169080	5.266584 $\pm$ 0.390220	0.385528	0	3
lambada_standard	30M	validation	accuracy	0.074279 $\pm$ 0.003618	0.071062 $\pm$ 0.002763	-0.003218	0	3
lambada_standard	30M	validation	perplexity	135.497543 $\pm$ 21.161153	210.832091 $\pm$ 71.159280	75.334548	0	3
lambada_standard	30M	validation	mean_target_nll	4.900933 $\pm$ 0.154586	5.305134 $\pm$ 0.387909	0.404201	0	3
lambada_standard	100M	test	accuracy	0.147358 $\pm$ 0.008834	0.150463 $\pm$ 0.006235	0.003105	2	3
lambada_standard	100M	test	perplexity	21.944875 $\pm$ 0.824615	21.292699 $\pm$ 0.434628	-0.652177	3	3
lambada_standard	100M	test	mean_target_nll	3.088069 $\pm$ 0.037225	3.058226 $\pm$ 0.020297	-0.029842	3	3
lambada_standard	100M	validation	accuracy	0.151434 $\pm$ 0.010014	0.161087 $\pm$ 0.005270	0.009653	3	3
lambada_standard	100M	validation	perplexity	20.610276 $\pm$ 0.910404	19.749080 $\pm$ 0.547300	-0.861196	3	3
lambada_standard	100M	validation	mean_target_nll	3.025144 $\pm$ 0.043930	2.982852 $\pm$ 0.027619	-0.042292	3	3

B-group metrics.

Hard-benchmark metrics.

Table 12: Aggregated hard-benchmark metrics across the three seeds.

Task	Scale	Metric	MHA mean $\pm$ SD	MHR mean $\pm$ SD	MHR-MHA	Wins	n
hellaswag	10M	acc	0.256556 $\pm$ 0.001880	0.257684 $\pm$ 0.000997	0.001129	3	3
hellaswag	10M	acc_norm	0.262929 $\pm$ 0.002150	0.264722 $\pm$ 0.000663	0.001792	2	3
hellaswag	30M	acc	0.266813 $\pm$ 0.001253	0.266614 $\pm$ 0.001092	-0.000199	0	3
hellaswag	30M	acc_norm	0.272024 $\pm$ 0.001322	0.272124 $\pm$ 0.002409	0.000100	1	3
hellaswag	100M	acc	0.277136 $\pm$ 0.002072	0.279725 $\pm$ 0.000434	0.002589	3	3
hellaswag	100M	acc_norm	0.291343 $\pm$ 0.002961	0.291044 $\pm$ 0.001597	-0.000299	2	3
piqa	10M	acc	0.535183 $\pm$ 0.004155	0.529742 $\pm$ 0.005477	-0.005441	0	3
piqa	10M	acc_norm	0.521763 $\pm$ 0.004318	0.510519 $\pm$ 0.009174	-0.011244	0	3
piqa	30M	acc	0.542437 $\pm$ 0.002372	0.550417 $\pm$ 0.009584	0.007980	2	3
piqa	30M	acc_norm	0.522488 $\pm$ 0.006889	0.534458 $\pm$ 0.011635	0.011970	2	3
piqa	100M	acc	0.593761 $\pm$ 0.009978	0.595575 $\pm$ 0.007545	0.001814	1	3
piqa	100M	acc_norm	0.585963 $\pm$ 0.008233	0.588683 $\pm$ 0.007903	0.002720	2	3
arc_easy	10M	acc	0.269006 $\pm$ 0.015328	0.267836 $\pm$ 0.008654	-0.001170	1	3
arc_easy	10M	acc_norm	0.239766 $\pm$ 0.007304	0.256140 $\pm$ 0.001754	0.016374	3	3
arc_easy	30M	acc	0.276023 $\pm$ 0.023559	0.271345 $\pm$ 0.003652	-0.004678	1	3
arc_easy	30M	acc_norm	0.256725 $\pm$ 0.017308	0.260819 $\pm$ 0.021936	0.004094	2	3
arc_easy	100M	acc	0.440936 $\pm$ 0.002680	0.437427 $\pm$ 0.004415	-0.003509	1	3
arc_easy	100M	acc_norm	0.360234 $\pm$ 0.007090	0.363158 $\pm$ 0.006077	0.002924	2	3

E.2 PER-SEED SECONDARY RESULTS

The following tables report the per-seed results for all nine task families. Each row gives the final checkpoint result for one model, scale, and seed.

Table 13: Per-seed arc_challenge accuracy results.

Scale	Model	Seed	acc	acc_norm
100M	MHA	42	0.197324	0.244147
100M	MHA	43	0.220736	0.247492
100M	MHA	44	0.220736	0.234114
100M	MHR	42	0.214047	0.234114
100M	MHR	43	0.227425	0.244147
100M	MHR	44	0.227425	0.234114
10M	MHA	42	0.193980	0.244147
10M	MHA	43	0.217391	0.284281
10M	MHA	44	0.214047	0.257525
10M	MHR	42	0.214047	0.240803
10M	MHR	43	0.200669	0.274247
10M	MHR	44	0.190635	0.247492
30M	MHA	42	0.200669	0.240803
30M	MHA	43	0.220736	0.244147
30M	MHA	44	0.210702	0.257525
30M	MHR	42	0.204013	0.210702
30M	MHR	43	0.190635	0.230769
30M	MHR	44	0.200669	0.224080

Table 14: Per-seed arc_easy accuracy results.

Scale	Model	Seed	acc	acc_norm
100M	MHA	42	0.443860	0.361404
100M	MHA	43	0.438596	0.352632
100M	MHA	44	0.440351	0.366667
100M	MHR	42	0.433333	0.370175
100M	MHR	43	0.436842	0.359649
100M	MHR	44	0.442105	0.359649
10M	MHA	42	0.256140	0.231579
10M	MHA	43	0.285965	0.245614
10M	MHA	44	0.264912	0.242105
10M	MHR	42	0.271930	0.257895
10M	MHR	43	0.273684	0.256140
10M	MHR	44	0.257895	0.254386
30M	MHA	42	0.292982	0.264912
30M	MHA	43	0.249123	0.236842
30M	MHA	44	0.285965	0.268421
30M	MHR	42	0.275439	0.285965
30M	MHR	43	0.270175	0.245614
30M	MHR	44	0.268421	0.250877

Table 15: Per-seed blimp_supplement overall-accuracy results.

Scale	Model	Seed	Overall accuracy	Examples
100M	MHA	42	0.570344	5218
100M	MHA	43	0.595795	5218
100M	MHA	44	0.569827	5218
100M	MHR	42	0.587775	5218
100M	MHR	43	0.605030	5218
100M	MHR	44	0.589331	5218
10M	MHA	42	0.582886	5218
10M	MHA	43	0.555388	5218
10M	MHA	44	0.572296	5218
10M	MHR	42	0.583166	5218
10M	MHR	43	0.560625	5218
10M	MHR	44	0.564588	5218
30M	MHA	42	0.546929	5218
30M	MHA	43	0.540752	5218
30M	MHA	44	0.571473	5218
30M	MHR	42	0.555547	5218
30M	MHR	43	0.582587	5218
30M	MHR	44	0.580101	5218

Table 16: Per-seed comps overall-accuracy results.

Scale	Model	Seed	Overall accuracy	Examples
100M	MHA	42	0.623166	49340
100M	MHA	43	0.629854	49340
100M	MHA	44	0.632732	49340
100M	MHR	42	0.592602	49340
100M	MHR	43	0.626409	49340
100M	MHR	44	0.626713	49340
10M	MHA	42	0.544447	49340
10M	MHA	43	0.528496	49340
10M	MHA	44	0.519497	49340
10M	MHR	42	0.540799	49340
10M	MHR	43	0.536826	49340
10M	MHR	44	0.542238	49340
30M	MHA	42	0.567714	49340
30M	MHA	43	0.558573	49340
30M	MHA	44	0.549676	49340
30M	MHR	42	0.564775	49340
30M	MHR	43	0.556668	49340
30M	MHR	44	0.547710	49340

Table 17: Per-seed entity_tracking overall-accuracy results.

Scale	Model	Seed	Overall accuracy	Examples
100M	MHA	42	0.288467	9483
100M	MHA	43	0.262507	9483
100M	MHA	44	0.323208	9483
100M	MHR	42	0.324787	9483
100M	MHR	43	0.321449	9483
100M	MHR	44	0.338761	9483
10M	MHA	42	0.241183	9483
10M	MHA	43	0.246563	9483
10M	MHA	44	0.142782	9483
10M	MHR	42	0.179152	9483
10M	MHR	43	0.268344	9483
10M	MHR	44	0.239527	9483
30M	MHA	42	0.378111	9483
30M	MHA	43	0.307876	9483
30M	MHA	44	0.405007	9483
30M	MHR	42	0.407819	9483
30M	MHR	43	0.350471	9483
30M	MHR	44	0.248045	9483

Table 18: Per-seed hellaswag accuracy results.

Scale	Model	Seed	acc	acc_norm
100M	MHA	42	0.278431	0.294762
100M	MHA	43	0.274746	0.289584
100M	MHA	44	0.278231	0.289683
100M	MHR	42	0.279526	0.289385
100M	MHR	43	0.279426	0.292571
100M	MHR	44	0.280223	0.291177
10M	MHA	42	0.258614	0.260904
10M	MHA	43	0.254929	0.262697
10M	MHA	44	0.256124	0.265186
10M	MHR	42	0.258813	0.264390
10M	MHR	43	0.256921	0.265485
10M	MHR	44	0.257319	0.264290
30M	MHA	42	0.267975	0.271261
30M	MHA	43	0.266979	0.271261
30M	MHA	44	0.265485	0.273551
30M	MHR	42	0.267477	0.271261
30M	MHR	43	0.266979	0.270265
30M	MHR	44	0.265385	0.274846

Table 19: Per-seed LAMBADA Standard results.

Scale	Model	Seed	Test acc	Test PPL	Test NLL	Val acc	Val PPL	Val NLL
100M	MHA	42	0.147681	21.5936	3.0724	0.158965	19.8012	2.9857
100M	MHA	43	0.138366	22.8870	3.1306	0.140070	21.5961	3.0725
100M	MHA	44	0.156026	21.3541	3.0612	0.155268	20.4335	3.0172
100M	MHR	42	0.152727	21.0142	3.0452	0.165537	19.2588	2.9580
100M	MHR	43	0.143412	21.7935	3.0816	0.155268	20.3395	3.0126
100M	MHR	44	0.155249	21.0704	3.0479	0.162456	19.6489	2.9780
10M	MHA	42	0.035901	375.7672	5.9290	0.034504	405.5814	6.0053
10M	MHA	43	0.037842	753.7046	6.6250	0.039433	818.3232	6.7073
10M	MHA	44	0.042694	487.7822	6.1899	0.042514	503.7286	6.2220
10M	MHR	42	0.050068	472.5370	6.1581	0.051345	484.8567	6.1839
10M	MHR	43	0.042694	518.2685	6.2505	0.049702	562.3982	6.3322
10M	MHR	44	0.044246	475.5015	6.1644	0.044362	528.7579	6.2705
30M	MHA	42	0.072385	112.6453	4.7242	0.078456	116.3114	4.7563
30M	MHA	43	0.068504	157.6199	5.0602	0.072294	158.1946	5.0638
30M	MHA	44	0.070056	128.8616	4.8587	0.072089	131.9867	4.8827
30M	MHR	42	0.067339	123.5239	4.8164	0.074143	128.6658	4.8572
30M	MHR	43	0.067533	246.8694	5.5089	0.070240	252.3256	5.5307
30M	MHR	44	0.065981	238.5214	5.4745	0.068803	251.5048	5.5275

Table 20: Per-seed openbookqa accuracy results.

Scale	Model	Seed	acc	acc_norm
100M	MHA	42	0.162000	0.298000
100M	MHA	43	0.168000	0.284000
100M	MHA	44	0.158000	0.276000
100M	MHR	42	0.160000	0.290000
100M	MHR	43	0.152000	0.284000
100M	MHR	44	0.160000	0.284000
10M	MHA	42	0.128000	0.156000
10M	MHA	43	0.126000	0.178000
10M	MHA	44	0.140000	0.174000
10M	MHR	42	0.140000	0.162000
10M	MHR	43	0.114000	0.172000
10M	MHR	44	0.128000	0.158000
30M	MHA	42	0.116000	0.182000
30M	MHA	43	0.126000	0.184000
30M	MHA	44	0.118000	0.156000
30M	MHR	42	0.104000	0.182000
30M	MHR	43	0.108000	0.158000
30M	MHR	44	0.124000	0.170000

Table 21: Per-seed piqa accuracy results.

Scale	Model	Seed	acc	acc_norm
100M	MHA	42	0.605005	0.595212
100M	MHA	43	0.585963	0.583243
100M	MHA	44	0.590316	0.579434
100M	MHR	42	0.593580	0.581066
100M	MHR	43	0.603917	0.596844
100M	MHR	44	0.589227	0.588139
10M	MHA	42	0.539717	0.520131
10M	MHA	43	0.531556	0.518498
10M	MHA	44	0.534276	0.526659
10M	MHR	42	0.534820	0.514690
10M	MHR	43	0.530468	0.516866
10M	MHR	44	0.523939	0.500000
30M	MHA	42	0.540805	0.529924
30M	MHA	43	0.541349	0.516322
30M	MHA	44	0.545158	0.521219
30M	MHR	42	0.544614	0.521763
30M	MHR	43	0.561480	0.544614
30M	MHR	44	0.545158	0.536997

F HYBRID RELATION

F.1 CONFIGURATION

Hybrid Relation uses a 12-layer, 480-dimensional decoder with eight heads of width 60 and $d_{\text{ff}} = 960$. Its layout is $(LLLF)^3$, with nine Linear Relation layers and three Full Relation layers. The vocabulary has 4,096 entries, context length is 2,048, and training budget is 450M tokens. The model has 31,967,571 parameters and uses micro/GA 16/4, BF16, AdamW, and 131,072 global tokens per update. This is a 30M-class configuration and is not geometry-identical to the 30M Full Relation model.

F.2 PER-SEED RESULTS

Table 22: Per-seed Hybrid Relation results.

Seed	Final NLL	Final BLiMP
42	1.274777	0.620134
43	1.283791	0.637343
44	1.275435	0.631582
Mean $\pm$ SD	1.278001 $\pm$ 0.005025	0.629687 $\pm$ 0.008760

F.3 SECONDARY RESULTS

The per-seed Hybrid BLiMP values are shown in Table 22. No additional Hybrid benchmark is used here.

G SYSTEMS BENCHMARK PROTOCOL

G.1 HARDWARE AND SOFTWARE

Systems measurements use an NVIDIA RTX 5090 GPU in BF16. The recorded software environment is CUDA 13.0, PyTorch 2.12.1+cu130, and Triton 3.7.1. Each campaign uses its formal prepared data and frozen model geometry.

G.2 TIMING AND ACCOUNTING

One-time initialization and JIT compilation are outside the reported interval. Warmup optimizer steps are excluded. Throughput uses complete optimizer steps, with the terminal partial token step excluded where the campaign reaches a non-integral final update. The denominator is the number of tokens consumed by the complete measured optimizer steps.

G.3 CAMPAIGN SETTINGS

The fixed-context Reference–FlashRelation campaign uses $T = 1024$ for all scales. The scale-matched FlashRelation–FlashAttention campaign uses $T = 1024, 2048, 4096$ for 10M, 30M, and 100M. The same-micro campaign uses $T = 1024$ for every scale, shared micro/GA at each point, and explicitly forced FlashAttention. Linear long-context scaling uses micro = 1, 131,072 effective tokens per update, five warmup steps, and ten measured complete steps.

H FLASHRELATION SYSTEMS RESULTS

H.1 SCALE-MATCHED FLASHRELATION AND FLASHATTENTION

FlashAttention is explicitly forced in these scale-matched production records. FlashRelation uses the final exact production dispatch for each head dimension.

Table 23: Scale-matched production FlashAttention and FlashRelation throughput; memory is allocated/reserved GiB.

Scale	T	FlashAttention tok/s	FlashRelation tok/s	FR/FA	FR micro/GA	FA micro/GA	FR alloc./res.	FA alloc./res.
10M	1024	841,833.930	672,838.258	0.799253 $\times$	64/2	32/4	10.26/11.31	5.38/5.88
30M	2048	361,793.115	307,146.176	0.848955 $\times$	16/4	16/4	9.72/10.15	10.03/10.45
100M	4096	107,783.242	82,332.578	0.763872 $\times$	4/8	4/8	17.32/18.24	17.71/18.63

H.2 COMPLETE SAME-MICRO SCALING

The tables below retain every micro point from the matched-micro campaign. FlashAttention is explicitly forced for all scales. Both methods use the same micro and gradient accumulation at each point, with $T = 1024$ and 131,072 effective tokens per update. OOM rows are retained as measured outcomes.

Table 24: Complete same-micro scaling for 10M; FlashAttention is forced.

Method	Micro/GA	tok/s	FR/FA	Δ	Alloc./res. GiB	Status
FlashAttention	1/128	30,706.567	0.658086	-0.341914	0.41/0.44	PASS
FlashRelation	1/128	20,207.558	0.658086	-0.341914	0.41/0.45	PASS
FlashAttention	2/64	63,454.977	0.997759	-0.002241	0.57/0.63	PASS
FlashRelation	2/64	63,312.789	0.997759	-0.002241	0.58/0.64	PASS
FlashAttention	4/32	121,247.542	0.693854	-0.306146	0.89/1.00	PASS
FlashRelation	4/32	84,128.151	0.693854	-0.306146	0.88/1.00	PASS
FlashAttention	8/16	242,675.755	0.672295	-0.327705	1.54/1.76	PASS
FlashRelation	8/16	163,149.784	0.672295	-0.327705	1.50/1.62	PASS
FlashAttention	16/8	460,243.223	0.790958	-0.209042	2.82/3.02	PASS
FlashRelation	16/8	364,033.195	0.790958	-0.209042	2.75/2.97	PASS
FlashAttention	32/4	835,322.853	0.725787	-0.274213	5.38/5.87	PASS
FlashRelation	32/4	606,266.514	0.725787	-0.274213	5.25/5.76	PASS
FlashAttention	64/2	769,418.196	0.877894	-0.122106	10.52/11.61	PASS
FlashRelation	64/2	675,467.374	0.877894	-0.122106	10.26/11.35	PASS
FlashAttention	128/1	754,729.251	0.877799	-0.122201	20.76/21.06	PASS
FlashRelation	128/1	662,500.761	0.877799	-0.122201	20.23/20.52	PASS

Table 25: Complete same-micro scaling for 30M; FlashAttention is forced.

Method	Micro/GA	tok/s	FR/FA	Δ	Alloc./res. GiB	Status
FlashAttention	1/128	21,715.706	0.739590	-0.260410	0.85/0.87	PASS
FlashRelation	1/128	16,060.716	0.739590	-0.260410	0.83/0.85	PASS
FlashAttention	2/64	42,208.004	0.759708	-0.240292	1.15/1.21	PASS
FlashRelation	2/64	32,065.762	0.759708	-0.240292	1.11/1.21	PASS
FlashAttention	4/32	82,115.368	0.770310	-0.229690	1.74/1.83	PASS
FlashRelation	4/32	63,254.298	0.770310	-0.229690	1.69/1.79	PASS
FlashAttention	8/16	161,224.613	0.745938	-0.254062	2.92/3.18	PASS
FlashRelation	8/16	120,263.553	0.745938	-0.254062	2.83/3.11	PASS
FlashAttention	16/8	322,442.432	0.764497	-0.235503	5.29/5.60	PASS
FlashRelation	16/8	246,506.254	0.764497	-0.235503	5.13/5.24	PASS
FlashAttention	32/4	388,949.491	0.866383	-0.133617	10.03/10.39	PASS
FlashRelation	32/4	336,979.375	0.866383	-0.133617	9.72/10.13	PASS
FlashAttention	64/2	352,213.572	0.888005	-0.111995	19.51/20.47	PASS
FlashRelation	64/2	312,767.346	0.888005	-0.111995	18.90/19.91	PASS
FlashAttention	128/1	—	—	—	—	OOM
FlashRelation	128/1	—	—	—	—	OOM

Table 26: Complete same-micro scaling for 100M; FlashAttention is forced.

Method	Micro/GA	tok/s	FR/FA	Δ	Alloc./res. GiB	Status
FlashAttention	1/128	15,954.945	0.759780	-0.240220	2.83/2.96	PASS
FlashRelation	1/128	12,122.251	0.759780	-0.240220	2.82/2.99	PASS
FlashAttention	2/64	30,462.198	0.756342	-0.243658	3.83/4.08	PASS
FlashRelation	2/64	23,039.846	0.756342	-0.243658	3.80/4.06	PASS
FlashAttention	4/32	58,932.391	0.777275	-0.222725	5.85/6.30	PASS
FlashRelation	4/32	45,806.677	0.777275	-0.222725	5.74/6.22	PASS
FlashAttention	8/16	115,222.124	0.770535	-0.229465	9.80/10.77	PASS
FlashRelation	8/16	88,782.650	0.770535	-0.229465	9.61/10.60	PASS
FlashAttention	16/8	133,917.125	0.841217	-0.158783	17.69/18.73	PASS
FlashRelation	16/8	112,653.302	0.841217	-0.158783	17.30/18.38	PASS
FlashAttention	32/4	—	—	—	—	OOM
FlashRelation	32/4	—	—	—	—	OOM
FlashAttention	64/2	—	—	—	—	OOM
FlashRelation	64/2	—	—	—	—	OOM
FlashAttention	128/1	—	—	—	—	OOM
FlashRelation	128/1	—	—	—	—	OOM

I LINEAR RELATION SYSTEMS AND LONG-CONTEXT SCALING

I.1 STEADY-STATE THROUGHPUT

Table 27: Frozen Linear Relation steady-state throughput.

Scale	T	Micro/GA	Steady-state tok/s	Raw mean tok/s
10M	1024	64/2	517,602	282,578
30M	1024	32/4	256,034	199,875
100M	1024	16/8	90,592	87,865

I.2 LONG-CONTEXT SCALING

Long-context runs use the production K_{L60} schedule with chunk 128, block 64, four warps, one stage, and row tile 1. Micro is fixed to 1 and the effective update size is 131,072 tokens. The $T = 65536$ point is a real OOM.

Table 28: Linear Relation long-context scaling.

T	Micro	GA	Status	tok/s	Allocated GiB
1024	1	128	PASS	10,129.47	1.140
2048	1	64	PASS	20,301.39	1.640
4096	1	32	PASS	39,990.24	2.648
8192	1	16	PASS	59,367.95	4.819
16384	1	8	PASS	52,894.96	8.827
32768	1	4	PASS	30,003.56	16.815
65536	1	2	OOM	—	—